

Linguaggio C: introduzione

Salvatore Filippone
salvatore.filippone@uniroma2.it

Federica Montesano
federica.montesano@uniroma2.it



- 1 Contatti
- 2 Informazioni Generali e Origini
- 3 Ambiente di Sviluppo e Installazione
- 4 Primo Programma in C
- 5 Il Processo di Compilazione
- 6 Sintassi Fondamentale: Keywords

- 1 Contatti
- 2 Informazioni Generali e Origini
- 3 Ambiente di Sviluppo e Installazione
- 4 Primo Programma in C
- 5 Il Processo di Compilazione
- 6 Sintassi Fondamentale: Keywords

Come contattarmi?

Via mail:

All'indirizzo `federica.montesano@uniroma2.it` con oggetto **[IngAlg]** nella mail.

Materiale del corso

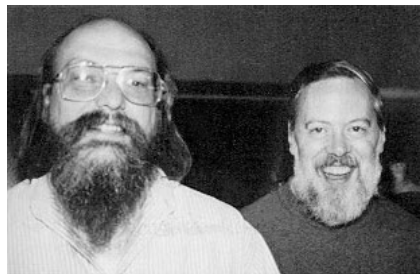
Slides ed esercizi verranno pubblicati sul sito <https://federicamontes.github.io/>.

- 1 Contatti
- 2 Informazioni Generali e Origini**
- 3 Ambiente di Sviluppo e Installazione
- 4 Primo Programma in C
- 5 Il Processo di Compilazione
- 6 Sintassi Fondamentale: Keywords

- Brian W. Kernighan, Dennis M. Ritchie
The C Programming Language (2nd edition)
Prentice Hall
ISBN: 978-0131103627

Le origini del linguaggio C

- Un linguaggio procedurale progettato da Dennis Ritchie ai Laboratori Bell tra il 1972 e il 1973 per scrivere strumenti per Unix
- Usato da Ken Thompson e Dennis Ritchie per riscrivere il Kernel Unix, originariamente scritto in Assembly
- Turing Award nel 1983
- Successo del sistema UNIX: raffinata selezione di alcune idee chiave e dalla loro elegante implementazione
- Genialità del sistema Unix è il suo framework, che consente ai programmatori di appoggiarsi al lavoro degli altri.



Roadmap

- 1 Contatti
- 2 Informazioni Generali e Origini
- 3 Ambiente di Sviluppo e Installazione**
- 4 Primo Programma in C
- 5 Il Processo di Compilazione
- 6 Sintassi Fondamentale: Keywords

Il software necessario

Compilatore

- Linux
- Mac
- Windows

Editor (cercate quello che va bene per voi):

- gedit
- Notepad++
- nano
- vim
- Emacs

Installazione di GCC: Linux e macOS

Linux (Ubuntu, Debian, Linux Mint, Kali)

Utilizzare il gestore di pacchetti apt dal terminale:

```
sudo apt update  
sudo apt install build-essential
```

macOS (Homebrew)

Installare prima il gestore pacchetti Homebrew e successivamente GCC:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"  
brew install gcc
```

Windows (MinGW Installer)

- ❶ Scaricare l'installer di **MinGW** (*Minimalist GNU for Windows*).
- ❷ Eseguire l'installer e selezionare il pacchetto relativo a **GCC**.
- ❸ Aggiungere la cartella MinGW\bin alla variabile d'ambiente PATH di sistema per abilitare il comando da terminale ^a.

^avedi <https://www.geeksforgeeks.org/cpp/installing-mingw-tools-for-c-c-and-changing-environment-variable/>

Verifica dell'Installazione

Verificare la corretta installazione del compilatore eseguendo nel terminale / prompt dei comandi:

```
gcc --version
```

Roadmap

- 1 Contatti
- 2 Informazioni Generali e Origini
- 3 Ambiente di Sviluppo e Installazione
- 4 Primo Programma in C**
- 5 Il Processo di Compilazione
- 6 Sintassi Fondamentale: Keywords

"Hello world"

```
1  #include <stdio.h>
2
3  int main(int argc, char *argv[])
4  {
5      int times=100;
6
7      printf("Hello, world! Welcome here %d times \n",times);
8      return(0);
9  }
```

"Hello world"

```
1  #include <stdio.h>  /* Importa un file */
2      /* questo è un commento */
3  int main(int argc, char *argv[]) /* qualunque programma esegue main */
4  {
5      int times=100;    /* Dichiaro una variabile */
6      /* Chiamata di una funzione */
7      printf("Hello, world! Welcome here %d times \n",times);
8      return(0); /* ritorno al sistema operativo */
9  } /* Fine di un blocco di istruzioni */
```

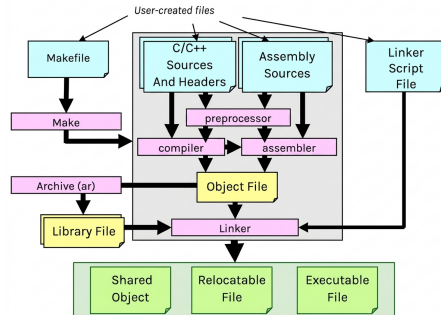
Roadmap

- 1 Contatti
- 2 Informazioni Generali e Origini
- 3 Ambiente di Sviluppo e Installazione
- 4 Primo Programma in C
- 5 Il Processo di Compilazione**
- 6 Sintassi Fondamentale: Keywords

Il processo di compilazione

Le 4 Fasi Principali

- 1 **Preprocessore** (.c → .i)
Espande le direttive (#include, #define).
- 2 **Compilatore** (.i → .s)
Traduce in codice assembly.
- 3 **Assembler** (.s → .o)
Genera il file oggetto in codice macchina.
- 4 **Linker** (.o + librerie → eseguibile)
Risolve i riferimenti ed unisce i moduli.



Automazione

make & Makefile: Automatizzano l'intero processo ricompilando solo i file sorgente modificati.

Compilazione Step by Step con GCC

1. Preprocessore (-E)

Espande le direttive (#include, macro) producendo il sorgente espanso:

```
gcc -E main.c -o main.i
```

2. Compilatore (-S)

Traduce il codice C espanso in linguaggio Assembly:

```
gcc -S main.i -o main.s
```

3. Assembler (-c)

Converte il codice Assembly in file oggetto (codice macchina):

```
gcc -c main.s -o main.o
```

4. Linker (-o)

Collega i file oggetto con le librerie di sistema per creare l'eseguibile:

```
gcc main.o -o main  
./main
```

Roadmap

- 1 Contatti
- 2 Informazioni Generali e Origini
- 3 Ambiente di Sviluppo e Installazione
- 4 Primo Programma in C
- 5 Il Processo di Compilazione
- 6 Sintassi Fondamentale: Keywords**

Definizione

Le **keywords** sono parole riservate del C con un significato predefinito per il compilatore e costituiscono gli elementi sintattici di base del linguaggio.

Regole Fondamentali

- **Non utilizzabili come identificatori:** non possono essere usate come nomi di variabili, funzioni o struct.
- **Case-sensitive:** sono scritte esclusivamente in minuscolo (e.g. `if` è una keyword, `IF` viene trattato come un normale identificatore).
- Lo standard **ANSI C (C89/C90)** definisce **32 parole chiave** (estese poi dagli standard moderni come C99, C11 e C23).